

Android 开发期末速通

目标：用最短时间抓住期末高频考点。适合考前 3 小时快速过一遍，考前 10 分钟背最后一节。

复习优先级

优先级	内容	复习目标
必背	四大组件、Activity 生命周期、Intent、布局、数据存储	能默写概念、特点、生命周期和代码模板
重点	Fragment、RecyclerView、网络请求、JSON、Kotlin 空安全	能解释用途，能看懂基础代码
加分	MVVM、Room、Retrofit、Jetpack	能写出组成和优点

第一章 Android 四大组件

1. Activity

作用： 一个 Activity 通常对应一个界面，负责和用户交互。

例子：

- 登录界面： `LoginActivity`
- 主界面： `MainActivity`
- 设置界面： `SettingsActivity`

生命周期

```
onCreate()
  ↓
onStart()
  ↓
onResume()
  ↓
运行中
  ↓
onPause()
  ↓
onStop()
  ↓
onDestroy()
```

高频场景

场景	执行顺序
第一次启动 Activity	<code>onCreate()</code> -> <code>onStart()</code> -> <code>onResume()</code>
按 Home 键回到桌面	<code>onPause()</code> -> <code>onStop()</code>
从后台重新回到 Activity	<code>onRestart()</code> -> <code>onStart()</code> -> <code>onResume()</code>
按 Back 键退出 Activity	<code>onPause()</code> -> <code>onStop()</code> -> <code>onDestroy()</code>
弹出透明/对话框式 Activity	通常执行到 <code>onPause()</code>

简答模板

Activity 是 Android 四大组件之一，表示一个用户界面，负责与用户进行交互。Activity 有完整的生命周期，常见回调包括 `onCreate()`、`onStart()`、`onResume()`、`onPause()`、`onStop()` 和 `onDestroy()`。

2. Service

作用： 在后台执行长期运行的任务，没有用户界面。

常见场景：

- 音乐播放
- 文件下载
- 后台定位
- 定时同步数据

生命周期

```
onCreate()  
onStartCommand()  
onDestroy()
```

易考点

- Service 没有界面。
- Service 运行在主线程，不等于自动开启子线程。
- 耗时任务仍然应该放到子线程、线程池或协程中执行。

简答模板

Service 是 Android 四大组件之一，用于在后台执行任务。它没有用户界面，适合音乐播放、文件下载等场景。
Service 的常见生命周期方法有 `onCreate()`、`onStartCommand()` 和 `onDestroy()`。

3. BroadcastReceiver

作用： 接收系统或应用发送的广播消息。

常见广播：

- 网络状态变化
- 电量变化
- 开机完成
- 应用内部通知

注册方式

注册方式	位置	特点
静态注册	<code>AndroidManifest.xml</code>	应用未启动时也可能接收部分广播
动态注册	Java/Kotlin 代码	生命周期更灵活，通常需要手动注销

动态注册常见代码：

```
registerReceiver(receiver, intentFilter);
unregisterReceiver(receiver);
```

简答模板

BroadcastReceiver 是广播接收器，用于接收系统或应用发出的广播，例如网络变化、电量变化等。它可以通过清单文件静态注册，也可以在代码中动态注册。

4. ContentProvider

作用： 在不同应用之间共享数据。

常见例子：

- 联系人
- 相册
- 短信
- 媒体库

核心特点

跨应用共享数据
统一 URI 访问
底层常配合 SQLite 使用

简答模板

ContentProvider 是 Android 四大组件之一，用于实现应用之间的数据共享。其他应用可以通过统一的 URI 访问其暴露的数据，常见例子包括联系人和媒体库。

第二章 Intent

作用： 实现组件之间的通信，常用于页面跳转和数据传递。

1. 显式 Intent

明确指定要启动的组件。

```
Intent intent = new Intent(MainActivity.this, SecondActivity.class);
startActivity(intent);
```

2. 隐式 Intent

不直接指定组件，而是通过动作、类别、数据等匹配目标组件。

```
Intent intent = new Intent(Intent.ACTION_VIEW);
intent.setData(Uri.parse("https://www.example.com"));
startActivity(intent);
```

3. 传递数据

发送数据：

```
Intent intent = new Intent(MainActivity.this, SecondActivity.class);
intent.putExtra("name", "Tom");
intent.putExtra("age", 20);
startActivity(intent);
```

接收数据：

```
String name = getIntent().getStringExtra("name");
int age = getIntent().getIntExtra("age", 0);
```

简答模板

Intent 是 Android 中用于组件通信的机制，可以实现 Activity 跳转、启动 Service、发送广播以及传递数据。Intent 分为显式 Intent 和隐式 Intent。

第三章 UI 界面开发

1. 常见布局

布局	特点
LinearLayout	线性布局，按水平或垂直方向排列
RelativeLayout	相对布局，根据控件之间的位置关系排列
ConstraintLayout	约束布局，适合复杂界面，当前常用
FrameLayout	帧布局，常用于 Fragment 容器或控件叠放

LinearLayout 方向：

```
android:orientation="vertical"
android:orientation="horizontal"
```

2. 常见控件

控件	作用
TextView	显示文本
EditText	输入文本
Button	按钮
ImageView	显示图片
ListView	旧式列表控件
RecyclerView	新式列表控件

3. RecyclerView

作用： 高效展示大量列表数据，通常用于替代 `ListView`。

核心组成

RecyclerView
Adapter
ViewHolder
LayoutManager

优点

复用 View
性能更高
支持复杂列表
支持动画
解耦数据和界面

简答模板

RecyclerView 是 Android 中用于显示列表数据的控件，常用于替代 ListView。它通过 ViewHolder 复用列表项视图，提高了列表显示性能，并且可以配合 LayoutManager 实现不同的布局效果。

第四章 Fragment

作用： Activity 中可复用的子界面模块。

常见用途：

- 底部导航的多个页面
- 平板双栏界面
- ViewPager 页面
- 模块化界面

特点

依附于 Activity
有自己的生命周期
可以复用
适合模块化开发

添加 Fragment

```
FragmentTransaction ft = getSupportFragmentManager().beginTransaction();
ft.add(R.id.container, new MyFragment());
ft.commit();
```

简答模板

Fragment 表示 Activity 中的一部分界面，必须依附于 Activity 使用。它有自己的生命周期，能够提高界面复用性，适合模块化开发和平板适配。

第五章 数据存储

1. SharedPreferences

作用： 轻量级键值对存储。

适合保存：

- 登录状态
- 用户名
- 设置项
- 少量配置数据

写入：

```
SharedPreferences sp = getSharedPreferences("user", MODE_PRIVATE);
sp.edit()
    .putString("name", "Tom")
    .putBoolean("login", true)
    .apply();
```

读取：

```
String name = sp.getString("name", "");
boolean login = sp.getBoolean("login", false);
```

易考点

- `apply()` 异步提交。
- `commit()` 同步提交，并返回是否成功。
- SharedPreferences 适合少量数据，不适合大量结构化数据。

2. SQLite

作用： Android 内置的轻量级关系型数据库。

常用类：

```
SQLiteOpenHelper  
SQLiteDatabase
```

SQLiteOpenHelper 两个高频方法：

```
onCreate()  
onUpgrade()
```

常见 SQL

建表：

```
CREATE TABLE student(  
    id INTEGER PRIMARY KEY,  
    name TEXT,  
    age INTEGER  
);
```

增：

```
INSERT INTO student(id, name, age)  
VALUES(1, 'Tom', 20);
```

删：

```
DELETE FROM student WHERE id = 1;
```

改：

```
UPDATE student  
SET name = 'Jack'  
WHERE id = 1;
```

查：

```
SELECT * FROM student;
```

简答模板

SQLite 是 Android 内置的轻量级关系型数据库，支持 SQL 语句，适合存储结构化数据。开发中通常继承

SQLiteOpenHelper 创建和升级数据库。

3. Room

作用： 对 SQLite 的封装，属于 Jetpack 组件。

核心组成：

组成	作用
Entity	数据表
DAO	数据访问接口
Database	数据库对象

简答模板

Room 是 Jetpack 提供的数据库框架，是对 SQLite 的封装。它通过 Entity 表示数据表，通过 DAO 定义数据库操作，通过 Database 提供数据库实例。

第六章 网络编程

1. HTTP 请求流程

客户端发送请求
服务器处理请求
服务器返回响应
客户端解析数据
更新界面

2. JSON

常见数据格式：

```
{  
  "name": "Tom",  
  "age": 20  
}
```

特点：

轻量
易读
常用于前后端数据交换

3. Retrofit

作用： Android 常用网络请求框架。

特点：

基于接口描述请求
代码简洁
常配合 Gson/Moshi 解析 JSON
常配合协程或 RxJava 处理异步

简答模板

Retrofit 是 Android 中常用的网络请求框架，可以通过接口和注解描述 HTTP 请求，并将服务器返回的数据转换为 Java/Kotlin 对象，简化网络编程。

第七章 Kotlin 高频考点

1. var 和 val

```
var a = 10
val b = 20
```

区别：

var： 变量，可重新赋值
val： 只读变量，不可重新赋值

2. 空安全

可空类型：

```
var name: String? = null
```

安全调用：

```
name?.length
```

非空断言：

```
name!!.length
```

Elvis 运算符：

```
val len = name?.length ?: 0
```

易考点

- `?` 表示变量可以为 `null`。
- `?.` 表示对象不为空时才调用。
- `!!` 表示强制认为不为空，可能抛出空指针异常。
- `?:` 表示为空时使用默认值。

3. when

类似 Java 的 `switch`，但更灵活。

```
when (x) {  
    1 -> println("one")  
    2 -> println("two")  
    else -> println("other")  
}
```

第八章 MVVM

MVVM 是常见 Android 架构模式。

层	全称	作用
M	Model	数据层，负责数据来源，如网络和数据库
V	View	界面层，负责显示 UI 和接收用户操作
VM	ViewModel	连接 View 和 Model，处理界面相关业务逻辑

结构:

```
View  
↓  
ViewModel  
↓  
Model
```

常见搭配:

```
ViewModel  
LiveData / StateFlow  
Repository  
Room  
Retrofit
```

优点

降低耦合
逻辑更清晰
便于维护
便于测试
适合复杂项目

简答模板

MVVM 是一种常见的软件架构模式，包括 Model、View 和 ViewModel。Model 负责数据，View 负责界面显示，ViewModel 负责处理界面逻辑并向 View 提供数据。MVVM 可以降低界面和业务逻辑之间的耦合，提高代码可维护性。

高频简答题模板

1. Android 四大组件是什么？

Android 四大组件包括 Activity、Service、BroadcastReceiver 和 ContentProvider。Activity 负责界面交互，Service 负责后台任务，BroadcastReceiver 负责接收广播，ContentProvider 负责跨应用共享数据。

2. Activity 生命周期有哪些？

Activity 常见生命周期方法包括 `onCreate()`、`onStart()`、`onResume()`、`onPause()`、`onStop()`、`onDestroy()`。当 Activity 从后台重新回到前台时，还会调用 `onRestart()`。

3. Intent 有什么作用？

Intent 用于 Android 组件之间的通信，可以实现页面跳转、启动服务、发送广播和传递数据。Intent 分为显式 Intent 和隐式 Intent。

4. SharedPreferences 和 SQLite 的区别？

SharedPreferences 是轻量级键值对存储，适合保存少量配置数据；SQLite 是关系型数据库，支持 SQL，适合保存大量结构化数据。

5. RecyclerView 为什么比 ListView 更常用？

RecyclerView 通过 ViewHolder 机制复用列表项视图，性能更高，并且支持更灵活的布局和动画效果，因此常用于替代 ListView。

6. Fragment 和 Activity 的关系？

Fragment 是 Activity 中的一部分界面，必须依附于 Activity 使用。一个 Activity 可以包含多个 Fragment，Fragment 可以复用并拥有自己的生命周期。

7. MVVM 的优点是什么？

MVVM 将数据、界面和界面逻辑分离，降低了代码耦合度，使项目结构更清晰，也更方便维护和测试。

易混点速记

易混点	区别
Activity vs Fragment	Activity 是完整界面；Fragment 是 Activity 中的子界面
Service vs 线程	Service 是组件，不等于子线程
SharedPreferences vs SQLite	前者存键值对；后者存结构化表数据
ListView vs RecyclerView	RecyclerView 更灵活，性能更好
显式 Intent vs 隐式 Intent	显式指定组件；隐式通过 action/data 匹配
apply() vs commit()	apply 异步无返回值；commit 同步有返回值
val vs var	val 只读；var 可变
?. vs !!	安全调用；强制非空

考前 10 分钟必背

四大组件：

Activity

Service

BroadcastReceiver

ContentProvider

Activity 生命周期：

onCreate

onStart

onResume

onPause

onStop

onDestroy

onRestart

Intent：

组件通信

页面跳转

数据传递

显式 Intent

隐式 Intent

布局:

LinearLayout

RelativeLayout

ConstraintLayout

FrameLayout

控件:

TextView

EditText

Button

ImageView

ListView

RecyclerView

RecyclerView:

Adapter

ViewHolder

LayoutManager

复用 View

性能高

存储:

SharedPreferences: 键值对、少量配置

SQLite: 关系型数据库、SQL、结构化数据

Room: Entity、DAO、Database

网络:

HTTP

JSON

Retrofit

Kotlin:

var 可变

val 只读

? 可空

? 安全调用

!! 非空断言

?: 默认值

when 条件分支

MVVM:

Model 数据层

View 界面层

ViewModel 界面逻辑层

优点: 低耦合、易维护、易测试

最后押题

最值得优先背的 8 个题：

1. Android 四大组件及作用。
2. Activity 生命周期及常见场景调用顺序。
3. Intent 的作用、分类和传值方式。
4. SharedPreferences 与 SQLite 的区别。
5. RecyclerView 的组成和优点。
6. Fragment 的作用和特点。
7. Kotlin 空安全符号 `?`、`?.`、`!!`、`?:` 的含义。
8. MVVM 的组成和优点。

把上面 8 题背熟，普通本科 Android 开发期末卷的基础分基本就稳了。